

## Concurrent Objects

**Invocation Notation:** *thread object.method(arguments)*. E.g.

*A q.enq(x)*

**Response Notation:** *thread object:result*. E.g. *A q: void*

An invocation is **pending** if it has no matching response.

**Sequential Histories:** Method calls of different threads do not interleave

**Well-Formed Histories:** Per-thread projections sequential

**Equivalent Histories:** Per-thread projections are the same.

A method call **precedes** another if response event precedes invocation event.

**Notation:** Given History  $H$ , method executions  $m_0$  and  $m_1$  in  $H$

- We say  $m_0 \rightarrow_H m_1$ , if  $m_0$  precedes  $m_1$
- Relation  $m_0 \rightarrow_H m_1$  is a **Partial order OR Total order if  $H$  is sequential.**

---

A sequential (multi-object) history  $H$  is **legal** if for every object  $x$ ,  $H|x$  is in the sequential spec for  $x$ . (per-object projection)

History  $H$  is **linearizable** if it can be extended to  $G$  by

- Appending zero or more responses to pending invocations
- Discarding other pending invocations
- So that  $G$  is **equivalent** to
  - **Legal sequential** history  $S$
  - where  $\rightarrow_G \subseteq \rightarrow_S$

**Composability Theorem:** History  $H$  is linearizable if and only if for every object  $x$ ,  $H|x$  is linearizable.

---

History  $H$  is **Sequentially Consistent** if it can be extended to  $G$  by

- Appending zero or more responses to pending invocations
- Discarding other pending invocations
- So that  $G$  is **equivalent** to a
  - **Legal sequential** history  $S$

**Sequential Consistency:** No need to preserve real-time order

- Cannot re-order operations done by the same thread
- Can re-order non-overlapping operations done by different threads (only keep relative order in the same thread)
- Not composable